CSE 476 Final Project Report

Sat Chidananda

5 December 2025

Project Overview

This report documents the design and implementation of an inference-time reasoning agent capable of solving multi-domain problems including Math, Coding, Planning and Common Sense. The agent employs a hierarchical routing strategy to dynamically select between Chain-of-Thought (CoT), Self-Consistency, and Direct Prompting techniques based on query complexity and resource constraints.

GitHub Repository: `https://github.com/163264128/NLP-Final-Project.git`

Agent Architecture

The core logic is encapsulated in the `ReasoningAgent` class defined in `src/agent.py`. The agent follows a cost-aware routing flow:

1. **Complexity Analysis:** The `is_question_complex` method evaluates the input. Questions containing keywords like "solve", "calculate", or "prove", as well as inputs exceeding 500 characters, are classified as complex.
2. **Strategy Selection:**
   - **Complex Queries:** Routed to the `ChainOfThought` solver to leverage step-by-step reasoning.
   - **Intermediate Queries:** If the API budget allows (call count < 8), the agent employs `SelfConsistency` to improve reliability via majority voting.
   - **Simple Queries:** `DirectSolver` is used for simple tasks or as a fail-safe mechanism if other methods yield no result.

Implemented Techniques

1. Chain-of-Thought (CoT)

Implemented in `src/techniques/cot.py`, this technique is the primary driver for mathematical and logical problems. The system prompt defines a "precise reasoning agent" and enforces a strict output format: `Final Answer: <answer>`. This constraints-based prompting ensures that the reasoning process does not bleed into the extracted answer.

2. Self-Consistency with Parallel Execution

Defined in `src/techniques/self_consistency.py`, this module enhances robustness by generating multiple reasoning paths for a single query.

- **Sampling:** It generates $n = 3$ independent outputs using a temperature of 0.7.
- **Optimization:** To mitigate latency, requests are executed in parallel using `concurrent.futures.ThreadPoolExecutor`.
- **Aggregation:** Answers are aggregated using `collections.Counter`, the final output is the most frequent consistent answer.

3. Direct Prompting

The `DirectSolver` (`src/techniques/direct_solver.py`) serves as the baseline and efficiency mechanism. It utilizes a concise system prompt instructing the model to provide *only* the final answer without intermediate steps.

Performance and Optimization

The infrastructure utilizes a robust `ModelAPI` client (`src/api_client.py`) with built-in error handling and timeout management. To handle the large volume of the test set (6,208 items), the submission script `src/generate_answers.py` implements a parallelized pipeline with 10 workers, capable of processing the entire dataset in approximately 1 hour while maintaining strict rate limits and thread-safe checkpointing.